

Yuktha Wellness

Multilingual RAG Health Assistant — Comprehensive Technical Overview

A client-readable and engineer-ready reference for a production health-assistant platform: architecture, retrieval, grounding, safety, multilingual behavior, conversation integrity, channel integration, verification evidence, operational lessons, current limitations, and verified present state.

Node.js | Express | Pinecone | BGE-M3 | BM25 | Cross-Encoder Reranking | Redis | Structured LLM Output | MongoDB | Web Chat | WhatsApp

Public technical edition · Evidence reconciled through 18 July 2026

1. Reading Guide and Evidence Policy

This document serves two audiences at once. Product leaders can read the executive summary, before-and-after evidence, safety model, current status, and limitations. Engineers can continue into the request lifecycle, module map, retrieval algorithms, cache boundaries, failure modes, verification gates, and representative traces.

Claims are based on the current integrated source, dated regression reports, live verification records, audit findings, and incident evidence. When a number changed over time, the document labels the date or scope instead of combining incompatible totals. When a capability exists in the backend but is not used by the verified frontend, those states are stated separately. The content is intentionally safe for broad professional review. It describes system behavior and engineering decisions without exposing private infrastructure, access material, private conversations, source-control identifiers, or private project administration.

Truth conventions used throughout:

- **Current implementation:** behavior confirmed in the integrated source and the most recent verification record.
- **Backend capability:** an implemented server path that may or may not be selected by the current web client.
- **Historical validation:** a dated test result retained as evidence of how the system evolved, not presented as the newest total.
- **Known limitation:** a verified boundary or missing gate, stated directly rather than hidden behind a broad production-ready claim.

2. Executive Overview

Yuktha Wellness is a production retrieval-augmented health assistant designed for a women's-health and metabolic-wellness platform. It accepts health questions through web chat and a controlled WhatsApp health lane, routes the request to the appropriate clinical domain, retrieves evidence from a curated vector knowledge base, and asks a structured language model to produce a concise user-facing response.

The system does not rely on a prompt alone for medical safety. Deterministic emergency and crisis checks run before retrieval or generation. The final generated answer is parsed against a strict schema, checked for confidence, and objectively compared with the retrieved evidence when required. Unsafe or unusable outcomes are replaced by controlled responses.

The retrieval path combines multilingual dense embeddings, per-domain BM25 keyword search, candidate deduplication, and self-hosted cross-encoder reranking. Six isolated knowledge namespaces cover PCOS, diabetes, MASLD/NAFLD, obesity, mental wellness, and home remedies. Cross-condition routing supports questions that span more than one domain.

The current model split is deliberate: GPT-4o-mini rewrites each health query into a self-contained retrieval query, while GPT-4.1-mini produces the structured final RAG answer. Keeping those roles separate prevents the lightweight rewrite choice from being mistaken for the final-answer model.

Conversation integrity is treated as part of safety. The assistant distinguishes an acknowledgement that answers one question from an acknowledgement following several questions or no question at all. Personal memory uses user-authored text as factual evidence; earlier assistant prose cannot re-enter retrieval and become an invented user fact.

Current high-level characteristics:

- Evidence-grounded answers: strict structured output, confidence gating, source identifiers, and objective cosine verification when citation evidence is absent or not independently strong.
- Pre-generation safety: category-specific medical emergency handling and a separate compassionate mental-crisis path run before the normal model pipeline.
- Multilingual retrieval: BGE-M3 produces 1024-dimensional vectors that align queries across many languages with the shared clinical corpus.
- Context-aware dialogue: topic-filtered history, provenance rules, bounded memory, and structural acknowledgement handling reduce stale-context leakage and unsupported assumptions.
- Graceful degradation: Redis, BM25, reranking, weather lookup, personal-memory writes, and query rewriting each have explicit failure behavior so an auxiliary failure does not automatically break the whole request.
- Verified channel separation: the backend supports JSON and server-sent-event responses, while the current verified web frontend intentionally calls the JSON endpoint; WhatsApp uses the same core pipeline behind a dedicated health-lane gate.

3. Product Purpose, Users, and Boundaries

The assistant is an educational and navigation layer. It helps users understand common health topics, prepare reasonable follow-up questions, receive safe lifestyle guidance, and recognize when urgent professional care may be needed. It is not a diagnosis engine, a prescribing system, or a replacement for a clinician.

The primary health scope includes reproductive and metabolic questions around PCOS, diabetes, fatty-liver conditions, obesity, mental wellness, diet, recipes, and curated home-remedy guidance. A semantic health-intent backstop also recognizes health-adjacent education questions that do not contain one exact keyword.

The assistant deliberately rejects clear general-knowledge trivia and unrelated requests. It also distinguishes a genuinely off-topic question from a short contextual reply such as 'barely slept' after the assistant has just asked about sleep. The conversation state, not message length alone, determines whether a short turn is meaningful.

Supported interaction classes:

- Health education: definitions, symptoms, risk factors, lifestyle considerations, and questions to discuss with a qualified professional.
- Diet and recipe guidance: condition-aware meal ideas and recipe formatting grounded in the relevant corpus.
- Home-remedy guidance: curated low-risk suggestions with contraindication awareness and an emergency exclusion.
- Mental-wellness support: warm educational support for ordinary distress and a separate deterministic path for crisis language.
- Image-assisted context: optional image analysis for supported face, skin, or tongue inputs, followed by the same routing and response controls used for text.
- Location-aware context: optional current coordinates can inform greeting, weather, and practical food context; failure to obtain them does not block chat.

Explicit non-goals:

- No autonomous diagnosis, prescription, or medication adjustment.
- No guarantee that a generated reply is clinically complete for an individual case.
- No use of a previous assistant statement as proof that the user reported a fact.
- No ordinary wellness advice when the deterministic safety layer identifies an emergency.
- No assumption that a backend feature is active in the current user interface merely because its route exists.

4. Starting Point and Engineering Constraints

The original chatbot was attached to a broader commerce application, so the engineering work had to improve the assistant without destabilizing unrelated account, catalog, order, content,

and checkout behavior. This shaped the preference for small modules, bounded interfaces, feature-specific tests, and surgical deployment practices.

Verified starting problems included:

- User-visible stack traces and inconsistent error responses.
- Answers with no dependable citation, confidence, or objective grounding gate.
- A knowledge-ingestion path that could mix product-oriented content with clinical retrieval.
- English-only embeddings and a single-condition knowledge scope.
- No dependable pre-model handling for medical emergencies or mental-health crises.
- In-process caches that disappeared across restarts and could not serve multiple workers.
- A retrieval wrapper that hid similarity scores and prevented direct threshold control.
- A reranker path that could fail silently and fall back without proving ranking separation.
- Conversation history that could reintroduce stale assistant text as if the user had said it.
- Short-message heuristics that confused valid follow-up answers with unrelated input.

The central engineering constraint was therefore not simply 'add an LLM.' The system needed explicit contracts between routing, retrieval, generation, memory, safety, channel ownership, and failure handling. Each contract needed a testable fallback.

5. Current Verified State

The present reference point is the integrated backend and frontend source verified on 18 July 2026, together with the live behavior checks completed during the July latency and routing repair. This section states what is implemented, what the current web client selects, and what remains conditional.

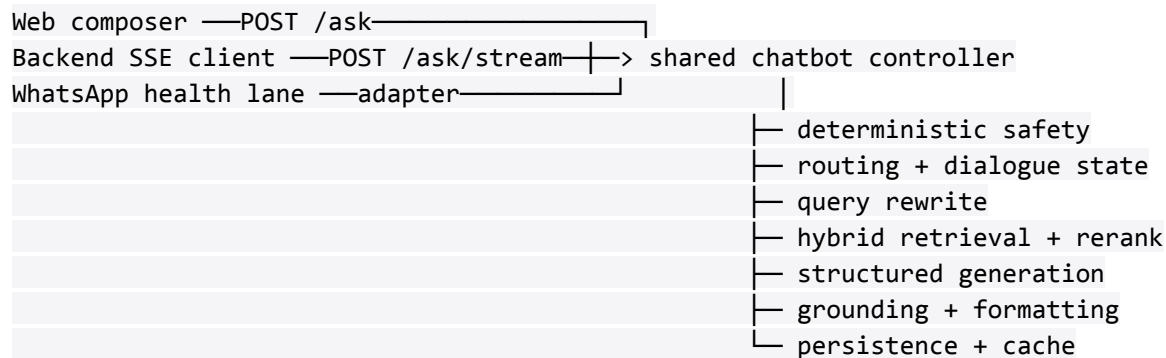
- Final answer generation: GPT-4.1-mini produces the strict structured RAG response.
- Retrieval-query rewriting: GPT-4o-mini produces the self-contained search query with a short, low-temperature call.
- Web request path: the verified frontend posts form data to /ask and includes fresh coordinates when available.
- Streaming status: the backend still exposes /ask/stream and shares the same pipeline and final grounding gate, but the verified frontend does not call that route.
- WhatsApp status: the AI is constrained to the Health Query lane; greetings, menus, forms, and non-health lanes remain owned by the messaging workflow.
- Retrieval status: BGE-M3, Pinecone, BM25, a 20-candidate rerank window, a five-result final set, and a 0.7 relevance threshold are active in the verified source.
- Caching status: response keys are versioned and scoped by user, normalized message or rewritten query, age, namespace, and page context; contextual turns are excluded from unsafe reuse.
- Product behavior: product lookup is an explicit user-request path, separated from the normal health answer; automatic insertion into ordinary health replies is not part of the verified frontend behavior.

- Frontend test status: the production build passes with existing static-analysis warnings, but the frontend source contains no automated test files.

6. System Topology

The assistant sits inside a larger Node.js application. Web and WhatsApp inputs enter through different channel adapters but converge on one chatbot controller. The controller owns request orchestration, while narrow services own retrieval, prompts, generation, safety, memory, image analysis, channel transport, and utility concerns.

Logical topology:



The channel adapters do not maintain separate clinical logic. The WhatsApp adapter invokes the same in-memory chatbot entry point used by the web controller, which keeps emergency detection, domain routing, multilingual retrieval, prompt rules, and grounding behavior aligned.

Primary data and service boundaries:

- MongoDB: user profile, bounded chat history, health-profile fields, and the surrounding commerce data.
- Pinecone: six condition knowledge namespaces plus per-user memory namespaces.
- Redis: rewrite, embedding, and response caches with time-limited entries and no-op failure behavior.
- Local transformer runtime: multilingual query embeddings and cross-encoder reranking.
- Language-model service: query rewrite, structured final answer, and optional supported image analysis.
- Messaging provider: verified webhook transport and outbound WhatsApp message delivery, separated from clinical logic.

7. End-to-End Request Lifecycle

The request path is intentionally ordered. Cheap deterministic decisions happen before network-heavy retrieval and generation. Context-sensitive gates occur before cache lookup, so a fast response cannot bypass dialogue integrity.

- 1 Validate text or supported image input
- 2 Start location/weather lookup and user lookup concurrently

- 3 Run medical-emergency detection
- 4 Run mental-crisis detection
- 5 Handle greeting and structural acknowledgement fast paths
- 6 Detect conditions, health intent, ambiguity, and off-topic input
- 7 Evaluate context-safe raw-response cache eligibility
- 8 Analyze an optional supported image
- 9 Rewrite the current message into a retrieval query
- 10 Query the condition corpus and user memory in parallel
- 11 Build topic-filtered history and explicit context blocks
- 12 Select a condition, recipe, or clarify prompt
- 13 Generate a strict structured answer
- 14 Apply emergency backstop, confidence gate, and grounding gate
- 15 Format, persist, update personal memory, and write eligible caches
- 16 Return JSON or emit SSE events through the same final authority

Each early return is deliberate. Emergency, crisis, greeting, ambiguous acknowledgement, off-topic fallback, symptom-intake, and explicit product requests do not need to traverse the full RAG path. This improves latency and reduces the number of states in which a model can affect the response.

8. Front-Door Routing and Fast Paths

Routing is layered rather than delegated to a single classifier. Deterministic patterns cover high-risk and high-confidence cases. Condition and health-intent helpers cover broad domain scope. A semantic classifier is used where language or conversational context makes a simple pattern unreliable.

8.1 Medical emergency path

The emergency detector runs before model or retrieval calls. It recognizes category families for cardiac, breathing, neurological, major bleeding or injury, severe allergy, pregnancy warning signs, overdose, and general acute danger. The response is selected by category, so a breathing emergency is not described with a generic unrelated template.

8.2 Mental-crisis path

Mental-crisis language is routed separately from ordinary medical emergencies. This preserves a compassionate tone and crisis-support information instead of returning a cold general ER message. Detector ordering is treated as product safety, not implementation trivia.

8.3 Greeting path

A greeting can be answered without RAG. If fresh location and weather information are available, they may enrich the greeting; lookup failures are caught and the greeting still completes.

8.4 Off-topic path

Clear trivia and unrelated attribution questions use a deterministic fallback before the expensive pipeline. False-positive guards keep health questions containing overlapping words such as 'paint', 'speech', or 'who' inside the health path when their full meaning is medical.

8.5 Ambiguous routine path

A request for a new 'routine' can mean diet, exercise, skin care, sleep, or general wellness. The assistant asks which kind before selecting a health namespace or generating a plan.

9. Dialogue State and Follow-Up Integrity

A short answer is not inherently ambiguous, and a long answer is not inherently self-contained. The current logic uses structural signals from the immediately preceding conversational turn. Operational channel markers are excluded before dialogue analysis.

Acknowledgement state machine:

- One preceding question: bind 'yes', 'no', or an uncertain acknowledgement to that question and provide an explicit acknowledgement context block to the generator.
- Several preceding questions: ask the user which question they meant; do not guess and do not infer that any proposed cause was reported as fact.
- No preceding question: ask what the acknowledgement refers to instead of routing it through health generation.

Context dependence is detected through actual acknowledgement patterns, anaphora such as 'that' or 'same', and elliptical openers such as 'what about' or 'how about'. The previous broad rule that treated most short messages as follow-ups was removed because it disabled safe caching for ordinary questions and produced wrong conversation state.

Uncertainty has a dedicated clarify mode. When a user cannot identify why they feel tired or cannot answer a symptom question, the assistant asks a small set of targeted questions rather than repeating the same remedy list. This clarify turn can bypass the ordinary RAG-similarity gate because it asks questions rather than making a clinical claim; the emergency backstop still applies.

10. Query Rewriting

Before retrieval, GPT-4o-mini rewrites the current message into one self-contained search query. The rewrite call uses at most the two most recent raw turns for pronoun and ellipsis resolution, a low temperature, and a short output limit. It returns only the search query.

The current verified website path always performs the rewrite. A previous attempt to skip it for apparently self-contained questions changed retrieved chunk identities for a meaningful subset

of queries because the rewrite normalizes terminology such as acronyms. That optimization was removed after retrieval comparison showed a quality regression.

Rewrite results are cached from the current message plus the limited disambiguation context. If the rewrite call fails or returns nothing, retrieval uses the raw current message instead of concatenating a noisy conversation transcript into the embedding.

Rewrite contract:

input = current user message + optional image description + up to 2 recent turns

output = one self-contained retrieval query, no explanation

failure = raw current message

cache = one-hour entry derived from message and disambiguation context

11. Hybrid Retrieval Pipeline

Retrieval combines semantic similarity with exact-term sensitivity. Dense search is strong at matching paraphrases and cross-language meaning; BM25 is strong when a clinical term, food name, symptom phrase, or acronym should materially affect rank. The two result sets are merged before a cross-encoder judges query-document relevance.

11.1 Dense search

The rewritten query is embedded into a 1024-dimensional BGE-M3 vector. The selected condition namespace returns the top 20 semantic matches with metadata. The vector client, index handle, embedder, and reranker are lazy singletons rather than reconstructed for every request.

11.2 Keyword search

Each clinical namespace has its own BM25 state so term frequencies are calculated within that domain instead of across unrelated corpora. The BM25 index is built from the same vector corpus during server warmup. Until a namespace is ready, keyword search returns an empty list and dense retrieval remains available.

11.3 Merge and deduplication

Dense results enter first with inline text and score metadata. BM25-only results are added by chunk identifier. Only BM25-only candidates inside the validated rerank window require a batched metadata fetch, avoiding a remote read for items that cannot affect the final answer.

11.4 Personal-memory reservation

For authenticated users, the condition query and personal-memory query run concurrently. Personal matches must exceed a stricter 0.75 similarity threshold and must contain a separately stored user message. Relevant personal candidates are reserved inside the same 20-item rerank window so they cannot be dropped solely because global results filled the window first.

11.5 Final context

The cross-encoder selects up to five documents. Each document contributes text and, where present, a visible source identifier. The service returns one context string plus the underlying vector chunk identifiers for response metadata and evaluation.

Current retrieval constants:

```
dense top-k           = 20
BM25 top-k           = 20
rerank candidate window = 20
final context documents = 5
dense fallback threshold= 0.70
personal memory threshold=0.75
```

12. Knowledge Base Design

The clinical corpus is divided by domain rather than stored as one undifferentiated pool. This improves relevance, makes evaluation easier to interpret, and limits accidental competition between unrelated topics. The documented ingestion snapshot contains 346 chunks across six namespaces.

- **pcos-knowledge:** PCOS definitions, symptoms, assessment context, lifestyle, diet, and related reproductive topics.
- **diabetes-knowledge:** glucose regulation, food guidance, lifestyle, risk, and general diabetes education.
- **masld-knowledge:** metabolic dysfunction-associated steatotic liver disease and legacy NAFLD terminology.
- **obesity-knowledge:** weight-management education, behavior, food, movement, risk, and sustainable habit guidance.
- **mental-wellness-knowledge:** ordinary emotional support and wellness education; crisis routing remains outside RAG.
- **home-remedies-knowledge:** curated low-risk remedy material with caution and contraindication context.

The condition router can return more than one domain. A question about PCOS with insulin resistance can draw from PCOS and diabetes evidence rather than forcing a false single-label decision. The controller selects a primary namespace for the main path while the condition context informs prompts and follow-up handling.

Ingestion uses overlapping recursive chunks to preserve local continuity. Deterministic chunk identifiers make repeated ingestion idempotent and allow retrieval outputs to be compared over time. A dimension check protects the 1024-dimensional index from incompatible vectors.

Knowledge governance boundaries:

- Clinical content and product-catalog content remain separate retrieval concerns.
- Emergency and crisis wording is controlled in code, not retrieved from ordinary corpus chunks.

- A source identifier is surfaced when available so the structured answer has something concrete to cite.
- Corpus changes should trigger the same golden retrieval comparison used for retriever changes.
- A high test count does not replace clinical review of newly added health material.

13. Multilingual Retrieval and Response Design

Multilingual behavior begins at retrieval, not only at the final prose layer. BGE-M3 maps supported Indian-language and English expressions into the same 1024-dimensional semantic space. This allows a user to ask in Hindi or transliterated language while retrieving relevant English clinical chunks.

The documented target set emphasizes Hindi, Tamil, Telugu, and Bengali, while the embedding model supports a much broader language range. The system also contains transliterated emergency and crisis patterns for high-risk phrases that should not wait for semantic retrieval. The final response language is guided by the user's language and prompt context. Retrieval source language and answer language are therefore decoupled: evidence can remain in a curated shared corpus while the response is written in the user's language.

Multilingual validation covers distinct failure classes:

- Routing accuracy: does the message reach the intended clinical domain or safety path?
- Cross-language retrieval: does the multilingual vector identify the same relevant evidence as the English equivalent?
- Transliteration handling: do common Latin-script forms of Indian-language phrases retain intent?
- Response language: does the assistant answer in a natural language consistent with the user rather than merely translating keywords?
- Safety consistency: do emergency and crisis phrases reach the same deterministic safeguards across language variants?

A documented 250-query evaluation snapshot included 220 reference questions and 30 Hindi transliteration questions. That historical evaluation is reported later with its date and scope; it does not imply complete coverage of every supported language, dialect, or code-switched form.

14. Cross-Encoder Reranking and Fallbacks

Dense similarity asks whether a query and chunk occupy a nearby semantic region. A cross-encoder instead evaluates the query and candidate text together, which is better suited to deciding whether a specific chunk directly answers the current question.

The primary reranker is a self-hosted MiniLM sequence-classification model. The implementation loads the tokenizer and model directly and reads the raw output logit. This was chosen after a higher-level classification wrapper both threw on paired inputs in the deployed

runtime and compressed scores so aggressively that relevant and irrelevant documents appeared identical.

A known relevant-versus-irrelevant probe demonstrated real separation after the repair: the relevant PCOS diet passage scored 6.39 while an unrelated geography passage scored -11.41. This kind of separation test is stronger than checking only that the reranker returns an array. The rerank window is fixed at 20 because smaller windows of 12 and 16 failed the golden retrieval gate. On the deployed CPU, scoring one pair at a time was faster for the representative 20-document workload than a fully padded batch. The implementation therefore follows measured hardware behavior rather than assuming batching is always faster.

Failure order:

local cross-encoder

→ optional hosted reranker when configured

→ dense-only top five above the 0.70 threshold

If every retrieval path produces no usable chunk, the generator does not receive fabricated context. The outer response logic supplies the controlled fallback instead of returning an empty body.

15. Structured Answer Generation

The final RAG answer uses GPT-4.1-mini. The request contains a condition-specific or recipe-specific system prompt, bounded relevant conversation turns, and an explicit user context assembled from separate tagged blocks. Temperature is kept moderate and output length is bounded differently for ordinary health replies and recipes.

The strict response schema requires four fields:

- reply: the user-facing answer.
- confidence: a number from 0 to 1 representing the model's grounding certainty.
- citations: an array of source identifiers supporting the reply.
- emergency_flag: a backstop signal for urgent medical escalation.

Strict schema mode prevents free-form wrapper text from bypassing the parser. A parse failure returns the controlled fallback. An empty reply also becomes the fallback, and output formatting occurs only after the final safety and grounding authority has accepted the answer.

The model-level emergency flag is a secondary safeguard. Deterministic emergency detection has already run before the model call, but an urgent signal found by generation still replaces the answer with the controlled emergency response.

The prompt layer adds behavioral constraints that are independent of the clinical domain: answer the asked question first, do not invent personal details, do not recite many remembered conditions, use at most one directly relevant memory, and avoid turning every reply into an interrogation.

16. Grounding, Citations, and the Empty-Citation Repair

Grounding is enforced after structured generation. The logic intentionally treats model-provided citations and objective semantic similarity as separate signals. This is more robust than assuming the model will populate a citation array perfectly on every otherwise grounded answer.

Current finalization logic:

parse failure	→ controlled fallback
model emergency flag	→ controlled emergency response
confidence missing or below 0.60	→ controlled fallback
citations present and confidence ≥ 0.90	→ accept without extra cosine call
citations absent	→ always run objective cosine grounding
cosine similarity below 0.55	→ controlled fallback
accepted structured reply	→ clean punctuation and formatting

The earlier rule rejected every answer with an empty citation array. In practice, the model sometimes omitted citation identifiers even when the answer was strongly supported by the retrieved context. This caused valid missed-period and food guidance to fall back incorrectly. The repaired logic does not simply trust citation-free answers. Instead, an empty citation array removes the high-confidence shortcut and forces the objective reply-versus-context cosine check. A citation-free answer can pass only when its semantic grounding reaches the same 0.55 threshold; self-rated confidence alone cannot bypass that check.

Retrieved chunks now expose source identifiers in the model-visible context when those identifiers are available. This improves the chance of a populated citation array without making citation formatting the single point of failure for an otherwise grounded answer.

Clarify turns are the narrow exception to ordinary cosine grounding. A question asking the user for sleep, stress, food, hydration, or illness context is not itself a clinical claim and may legitimately have low similarity to remedy chunks. That bypass is explicit, turn-scoped, and retains the emergency backstop.

17. Medical Safety Architecture

Health-assistant safety is implemented as defense in depth. Deterministic pre-model detection handles clear emergencies. The structured answer includes an emergency backstop. Prompt rules limit claims and personal assumptions. Grounding filters unsupported output. Channel-specific product behavior is kept separate from crisis and emergency flows.

17.1 Category-specific emergency responses

The emergency helper returns a response matched to the detected category: cardiac, breathing, neurological, bleeding or trauma, severe allergy, pregnancy, overdose, or general acute danger. The wording emphasizes urgent care and suppresses home-remedy suggestions.

17.2 Generalized neurological detection

Stroke-like language is detected through combinations of acute onset, one-sided facial weakness or numbness, and new speech difficulty. This generalizes beyond one report while reducing false positives from benign historical or therapy-related mentions.

17.3 Cardiac false-positive control

Cardiac handling uses symptom and location combinations, radiation toward arm or jaw, and red-flag symptoms around palpitations. Benign heart-health phrases and isolated heartburn wording are guarded so the detector does not escalate every occurrence of the word 'heart'.

17.4 Persistence of flagged turns

Safety-triggered turns can be recorded for safeguarding and audit, but the response itself does not wait for retrieval or generation. If persistence fails, the urgent user-facing action remains the priority.

17.5 Safety limitations

Pattern coverage is conservative, not exhaustive. Novel phrasing, unsupported languages, sarcasm, typos, and indirect descriptions can still escape deterministic detection. The model emergency flag is a backstop, not proof of complete recall. Safety patterns therefore require ongoing review with false-positive and false-negative cases.

18. Mental-Wellness and Crisis Routing

Ordinary low mood, anxiety, overwhelm, loneliness, breakup distress, panic wording, poor focus, and emotional exhaustion are treated as health-related support topics rather than rejected as trivia. The semantic classifier can route natural emotional language into the mental-wellness domain even when no exact pattern is present.

Active or passive suicidal ideation, self-harm intent, severe hopelessness, and common Hindi transliterations use a separate crisis detector. That path returns compassionate support and appropriate crisis resources. A completed overdose remains an acute medical emergency because immediate physical care is required.

The ordering was corrected after crisis phrases were found inside the broader emergency detector and therefore received a cold generic urgent-care reply. Pure ideation patterns were moved to the crisis layer, while acute medical consequences remained in the medical path.

Mental-wellness response principles:

- Acknowledge emotion without claiming knowledge of the user's personal history.
- Keep crisis behavior deterministic and separate from ordinary retrieval.
- Do not append product suggestions to crisis or emergency responses.

- Preserve true off-topic rejection for unrelated entertainment or trivia questions.
- Treat supportive conversation quality as testable behavior, not only prompt style.

19. Personalization and User-Memory Provenance

Authenticated users can contribute structured health-profile context such as conditions, symptoms, lab values, and medications. A separate personal vector namespace stores embedded user messages so directly relevant prior information can be recovered during later questions.

The original personal-memory record embedded a combined 'User + Bot' string. This allowed an earlier assistant statement to return through vector search and be mistaken for something the user had reported. The current record embeds only the user-authored message. Assistant output may remain separate metadata for traceability but is not injected as factual evidence. Legacy records are handled conservatively. If a record has a separate original user query, the retriever rebuilds the memory text from that field. If the record contains only a mixed legacy text blob, it is omitted rather than risking a false personal claim.

A 0.75 relevance threshold filters personal matches before they enter the candidate pool. This threshold was validated against a reproduced leak: a stale breastfeeding memory scored 0.687 for an intermittent-fasting query and was filtered, while a relevant diabetes memory scored 0.777 and remained available.

Similarity alone is not sufficient. Related memories about eating, mood, and diabetes can all score highly for a fasting question. The prompt therefore answers the current question first and uses at most one directly relevant remembered detail instead of listing every related condition or opening with an interrogation.

20. Context Assembly and History Filtering

The generator receives context in explicitly separated blocks. This reduces ambiguity about whether a sentence came from the current user, a stored profile, retrieved evidence, conversation history, or a dialogue-state note.

```
<user_query>current message</user_query>
<user_profile>structured user-provided health fields</user_profile>
<rag_context>retrieved clinical and permitted personal evidence</rag_context>
<conversation_history>topic-relevant ordered turns</conversation_history>
<acknowledgement_context>one-question binding when
applicable</acknowledgement_context>
<ambiguity_context>clarification guidance when applicable</ambiguity_context>
```

Each saved conversational turn can include a BGE-M3 embedding. For a new message, history turns above a 0.5 cosine threshold are selected, capped at six, and restored to chronological order. If too few old turns have embeddings or the new embedding fails, the system falls back to the most recent six turns.

The persisted chat history is bounded to the latest 40 entries. This prevents unbounded document growth, while the topic filter prevents the generator from receiving all 40 on every request. Operational WhatsApp mode markers are excluded from conversational turn analysis. The context layer also starts the current-turn embedding before generation and waits for it during persistence. That overlap reduces visible latency without weakening save reliability.

21. Performance, Caching, and Latency Repair

Performance work focused on measured stage latency and correctness-preserving reuse. The system records stage durations for request preparation, cache lookup, query rewrite, retrieval, history preparation, generation, persistence, and cache writes. Retrieval adds its own embedding, vector-query, merge/fetch, and rerank substages.

21.1 Cache layers

- Rewrite cache: maps the current message and limited disambiguation context to the rewritten retrieval query.
- Query-embedding cache: maps normalized retrieval text to its BGE-M3 vector.
- Reply-embedding cache: reuses reply vectors needed by the cosine grounding gate.
- Raw-response cache: serves exact self-contained repeats before rewrite and retrieval when the dialogue state makes reuse safe.
- Rewritten-query response cache: serves eligible repeated retrieval queries after rewrite but before retrieval and generation.

21.2 Context-safe response keys

Response keys include a version, user scope, normalized message or rewritten query, age, condition namespace, and page context. Image requests, explicit product requests, recovered-condition follow-ups, casual contextual turns, and active symptom-intake flows are excluded. This prevents a fast cache hit from replaying an answer that belongs to a different user or conversational state.

21.3 Safe parallelism

Independent work is overlapped: user lookup and weather lookup; category preparation and query rewriting; condition-corpus and personal-memory vector queries; history embedding and answer generation; and eligible cache writes. Promise settlement is used where one optional dependency should not cancel the other result.

21.4 July latency evidence

A controlled seven-case live suite measured 6.956 seconds for a fresh uncached PCOS question, 8.716 seconds for a different uncached question in the same domain, 5.666 seconds

for a contextual reply, and 0.161 seconds for an exact Redis repeat. Emergency, off-topic, and ambiguous-acknowledgement fast paths completed between 0.146 and 0.214 seconds in that run.

An independent real-history web-chat run later measured two normal health responses at 9.442 and 6.547 seconds, while emergency, trivia, and clarification paths remained below one second. These are observed samples, not a guaranteed service-level bound; external model, vector, network, and hardware variance still applies.

21.5 Cache failure behavior

Every cache read returns a miss when Redis is unavailable, and every cache write becomes a failed no-op. The request continues through the ordinary path. This preserves correctness at the expense of latency and repeated work, which is the appropriate degradation mode for an optional cache.

22. Web Chat Channel

The web composer sends multipart form data so text, optional images, page context, and fresh coordinates can share one request contract. It blocks overlapping submissions, accepts authentication from the supported browser storage locations, and keeps the widget usable when location is denied or a request fails.

22.1 Current frontend behavior

The verified current frontend posts to /ask. Source and production-bundle checks confirm that /ask/stream is absent from the selected frontend path, while /ask and fresh latitude/longitude fields are present. This is the current user-facing behavior.

22.2 Backend streaming capability

The backend separately retains /ask/stream. That route is a thin wrapper over the same chatbot controller: it sets stream mode, emits token events, buffers the full structured output, and runs the same final safety and grounding function as the JSON route. If finalization replaces the streamed draft, the stream can emit an authoritative replacement.

The presence of the route is therefore an available server capability, not evidence that the current web client uses it. Future reactivation should include a frontend test for replacement events, error handling, overlapping requests, and parity with /ask before the user interface switches endpoints.

22.3 Image input

Supported uploads are analyzed before retrieval. Unsupported images return a clear validation response. Image requests are excluded from response caches because two uploads with the

same text are not equivalent. The image description becomes part of the rewritten query and the final context.

22.4 Location and weather

The frontend attempts a fresh coordinate read and falls back to stored coordinates when appropriate. On the server, reverse location and weather lookup run concurrently with user lookup. Weather errors are caught and the request proceeds with location marked unavailable.

23. WhatsApp Health-Lane Integration

WhatsApp is a channel adapter over the same assistant core, not a second health bot. The adapter verifies inbound webhooks, acknowledges quickly, parses the provider payload, decides whether the AI owns the current lane, then invokes the shared chatbot entry point asynchronously.

23.1 Fast acknowledgement

Inbound webhooks must be acknowledged before a full RAG answer can finish. The controller returns success first, then performs routing and sends the outbound answer. After acknowledgement, processing is enclosed in its own error boundary so it never attempts to reuse the completed HTTP response.

23.2 Single-orchestrator ownership

The messaging workflow owns greetings, menus, forms, webinar flows, waitlists, and other non-health interactions. The AI owns free-text health questions only after the user enters the Health Query lane. Tapping another menu option switches the stored mode off, and strict default behavior stays silent when there is no active health lane.

23.3 Mode markers

Health-lane on/off state is stored as lightweight operational markers in existing history rather than requiring a new user schema. The most recent marker wins until its timeout expires. Markers use an empty query, do not count as health answers, and are ignored by conversational-memory helpers.

23.4 One-time notices

Channel notices that should appear once are also represented by an operational marker. The controller scans the full history for that marker, so later health-mode changes cannot cause the same notice to fire again.

23.5 Shared health behavior

Once a message is accepted as a health question, the channel calls the same emergency, crisis, condition, rewrite, retrieval, generation, and grounding pipeline as the website. This avoids a fork in clinical behavior and allows the same regression suites to protect both channels.

WhatsApp ownership decision table:

Health Query button	→ health mode ON, prompt user for question
Any other menu button	→ health mode OFF, messaging workflow continues
Greeting in orchestrated mode	→ silent from AI
Form or non-text step	→ silent from AI
Free text while ON	→ shared health pipeline
Free text while OFF	→ silent from AI
Human-support request	→ direct support handoff response

24. Module and File Architecture

The implementation is organized around one orchestration controller and focused services. The following map is useful when reviewing behavior, planning tests, or deciding where a change belongs. Paths are repository-relative and contain no environment-specific location.

24.1 Entry points and channel adapters

- `controllers/chatbot/chatbot.controller.js` — shared orchestration, early exits, cache policy, context assembly, persistence, JSON/SSE adapters, and in-memory entry point for other channels.
- `routes/v1/chatbot/chatbot.routes.js` — authenticated `/ask` and `/ask/stream` route declarations with upload handling.
- `controllers/whatsapp/whatsapp.controller.js` — webhook lifecycle, channel ownership, health-mode state, one-time notices, and shared-pipeline invocation.
- `services/whatsapp/interakt.service.js` — signature verification, payload parsing, button-title interpretation, text conversion, outbound transport, and human-support helpers.
- `routes/v1/whatsapp/whatsapp.routes.js` — messaging webhook route surface.

24.2 Retrieval and model services

- `services/rag.service.js` — query rewrite contract, rewrite cache, raw-message fallback, and retrieval service wrapper.
- `utils/ragRetriever.js` — dense/personal queries, BM25 merge, candidate window, metadata fetch, rerank order, relevance thresholds, source context, and retrieval telemetry.

- `utils/bgeEmbedder.js` — multilingual embedding singleton, 1024-dimensional contract, and boot warmup.
- `utils/bm25.js` — per-namespace keyword indexes, corpus loading, readiness state, and top-k scoring.
- `utils/reranker.js` — self-hosted cross-encoder lifecycle, raw-logit scoring, warmup, and non-fatal failure behavior.
- `services/llm.service.js` — structured schema, model request, JSON parsing, emergency backstop, confidence/grounding gates, and safe streaming finalization.

24.3 Routing, safety, and prompts

- `utils/safety.js` — medical emergency families, generalized cardiac/neurological logic, category messages, crisis patterns, and detector exports.
- `utils/conditionRouter.js` — multi-condition detection and mapping into six knowledge namespaces.
- `utils/healthIntentRouter.js` — broad health-scope and deterministic off-topic logic.
- `services/intentClassifier.service.js` — semantic follow-up, emotional-topic, and multilingual uncertainty classification.
- `services/followUpContext.service.js` — ambiguity context for continuing conversations without repeating or guessing.
- `services/prompt.service.js` — shared behavioral rules plus condition, recipe, and clarify prompt construction.
- `utils/regex.js` — bounded symptom, recipe, and health-education language families used by routing.

24.4 Memory, cache, and observability

- `utils/chatMemory.js` — acknowledgement state, operational-marker filtering, topic-relevant history, stage detection, and turn embeddings.
- `services/personalization.service.js` — structured profile context and user-message-only personal-vector writes.
- `utils/cache.js` — normalized hashing, lazy Redis access, JSON reads/writes, and safe no-op degradation.
- `utils/latencyTrace.js` — request identifiers, per-stage timing, total duration, outcome labels, and structured logs.

24.5 Optional contextual services

- `services/image.service.js` — supported image analysis and invalid-image classification.
- `utils/location.js` — location/weather lookup and graceful failure behavior.
- `utils/greetingCheck.js` — greeting detection and dynamic greeting construction.
- `utils/categoryMatcher.js` — page/product category interpretation used outside the core clinical evidence path.
- `utils/formatter.js` — post-gate user-facing formatting and dynamic heading cleanup.

25. Data Flow and Persistence Boundaries

25.1 Guest request

A guest can traverse safety, routing, retrieval, and generation without creating personal vector memory. Guest response caches use a guest scope so they cannot share authenticated-user context.

25.2 Authenticated request

An authenticated request can read structured health-profile fields, topic-filtered conversation history, and a relevance-gated personal namespace. On completion, the latest turn is persisted within the bounded history and the user-authored message is embedded into personal memory through a non-blocking helper.

25.3 Safety-triggered request

Emergency and crisis requests return before normal retrieval. The user-facing response is deterministic. A flagged record may be stored for safeguarding, but persistence is not allowed to delay the urgent response.

25.4 Cached request

A safe raw-cache hit skips rewrite, retrieval, and generation. A safe rewritten-query cache hit skips retrieval and generation after the rewrite. Both still occur only after medical safety, crisis, greeting, acknowledgement, and routing decisions that protect the current dialogue state.

25.5 Explicit product request

Product lookup queries the application catalog after health intent has been interpreted. It returns a separate list with application routes rather than presenting catalog text as clinical evidence. Product requests are not response-cacheable because inventory and matching can change independently of a health answer.

26. Observability and Failure Semantics

Observability focuses on stage attribution. A slow request is not described merely as 'the model is slow' or 'Redis is slow.' Structured timing separates user/weather preparation, cache lookup, query preparation, retrieval, history work, generation, persistence, and cache writes. The retriever exposes its own substages.

Representative outcomes recorded by the request trace:

- response — a normal non-cached answer completed.
- cache-hit — a safe response cache supplied the answer.
- clarification — dialogue state returned a deterministic clarify response.
- error — the controller reached its protected error boundary.

The request identifier is short and random, allowing stage logs from one request to be correlated without placing the full user message in every timing line. Retrieval telemetry reports counts and timing rather than dumping the entire evidence corpus.

Failure semantics by dependency:

- Weather lookup failure: continue without weather context.
- Redis failure: treat reads as misses and writes as no-ops.
- Query rewrite failure: use the raw current message.
- Personal-memory query failure: continue with the condition corpus.
- BM25 not ready: continue with dense candidates.
- Local reranker failure: try the optional secondary path, then dense-only filtering.
- Retrieval failure: return no evidence and let the response layer produce a controlled fallback.
- Generation or parse failure: return the controlled fallback.
- Grounding-embedding failure: log and continue under the existing confidence/citation state; this is a known availability-versus-strictness tradeoff.
- Personal-memory write failure: return the answer and log the non-fatal persistence issue.

27. Operational Cost Reduction

Operational cost reduction was achieved by removing unnecessary remote work, reusing safe results, and moving ranking onto the application host. This section reports engineering mechanisms without currency figures.

27.1 Self-hosted reranking

The primary reranking step uses a local cross-encoder. Normal requests therefore do not require a hosted rerank call. A secondary hosted option remains a resilience path when configured, not the default ranking dependency.

27.2 Rewrite reuse

Rewrites are cached for an hour using the current message and limited disambiguation context. Repeated equivalent turns avoid another rewrite call without sacrificing the retrieval quality that was lost when rewrites were skipped entirely for apparently self-contained questions.

27.3 Embedding reuse

Normalized query embeddings, reply-grounding embeddings, and saved turn embeddings are reused. The system does not re-embed the same normalized text for every layer and every repeated request.

27.4 Full-response reuse

Context-safe raw and rewritten-query caches eliminate rewrite, vector retrieval, reranking, and final generation for eligible repeats. The 0.161-second observed exact repeat demonstrates both latency and resource reduction.

27.5 Bounded candidate work

The reranker evaluates at most 20 candidates and fetches metadata only for BM25-only items that fall inside that window. This avoids vector-store reads and local inference for candidates that cannot influence the final five.

27.6 Deterministic fast paths

Emergency, crisis, greeting, clear trivia, ambiguous acknowledgement, and routine clarification paths avoid the ordinary RAG sequence. This removes unnecessary rewrite, retrieval, rerank, and generation work while making high-confidence behavior more predictable.

27.7 Safe rather than aggressive optimization

An optimization is retained only when it preserves retrieval and dialogue integrity. The system restored always-on query rewriting after chunk comparison proved that regex-based skipping changed evidence. Candidate windows below 20 were rejected by the golden gate. Cost reduction is therefore subordinate to verified answer quality.

28. Security and Privacy Boundaries

The assistant operates inside an authenticated application and handles potentially sensitive health context. Security review therefore covers both ordinary web risk and AI-specific cross-user or context-leak risk.

28.1 Authentication and upload boundary

Chat routes use the existing authentication middleware, and image input passes through bounded upload handling before analysis. The controller rejects a request with neither text nor an image. Supported image analysis is narrowly scoped rather than treating every upload as useful clinical evidence.

28.2 Webhook integrity

The messaging webhook verifies the provider signature over the raw body and rejects missing or mismatched signatures when verification is enabled. The raw-body capture exists only to support verification before JSON normalization changes the byte representation.

28.3 Cross-user cache isolation

Authenticated response cache keys include user scope, and contextual turns are excluded from unsafe caching. This prevents an answer personalized for one user from being reused for another user merely because the visible message text is the same.

28.4 Personal-data provenance

Only separately stored user-authored text is permitted to re-enter personal vector retrieval as factual evidence. Mixed legacy records are omitted when provenance cannot be reconstructed. Prompt rules forbid stating pregnancy, breastfeeding, diagnosis, symptoms, or past experience as fact unless the user actually provided it.

28.5 Error exposure

User-facing errors are controlled and do not intentionally expose stack traces. Internal logging records the technical failure for diagnosis while the response body remains concise.

28.6 Review coverage

A documented security review reported no findings at the chosen high-confidence threshold across injection, server-side request misuse, path traversal, authentication bypass, weak cryptographic usage, response-cache cross-user leakage, embedding-cache poisoning, and stack-trace exposure. This is a point-in-time review, not a substitute for dependency scanning and recurring assessment.

29. Test Strategy

Verification is layered because a health RAG system can pass ordinary unit tests while still retrieve the wrong evidence, route a crisis incorrectly, leak context, or behave differently through another channel. The strategy therefore includes static syntax checks, deterministic unit-style scripts, retrieval evaluation, build checks, controlled API tests, and live behavior samples.

29.1 Syntax and static gates

- Node syntax checks for every changed runtime module and test script.
- Whitespace and diff hygiene before review.

- Static checks that JSON and SSE routes share the same controller and finalizer.
- Frontend bundle checks for selected endpoint and fresh coordinate fields.

29.2 Deterministic behavior suites

- Emergency categories, generalized neurological cases, and benign false-positive guards.
- Health-intent routing, off-topic trivia, emotional-topic routing, and semantic uncertainty.
- One-question, multi-question, and zero-question acknowledgement handling.
- Response-cache eligibility, user/context key isolation, and Redis-down behavior.
- Structured output parsing, confidence fallback, citation handling, and cosine grounding.
- Personal-memory provenance, relevance filtering, and prompt behavior.
- WhatsApp signature, ownership, health-mode, one-time notice, and shared-pipeline behavior.

29.3 Retrieval gates

The golden retrieval suite compares the actual chunk identifiers returned for 30 fixed questions across all six namespaces. The gate measures recall against the validated set and catches changes that still produce fluent but differently grounded answers.

29.4 Controlled live tests

Controlled live checks use dedicated temporary identities, exact cleanup, endpoint health checks, process stability, and stage latency logs. Exact identifiers are deleted individually; broad range deletion is prohibited because a past cleanup command removed one pre-existing record.

29.5 Build verification

The current integrated frontend produces a successful production build. Existing static-analysis warnings remain visible, and the absence of frontend test files is reported rather than treating a successful build as complete UI behavior coverage.

30. Test Evidence and How to Read the Totals

Test counts changed as fixes and new suites were added. The totals below are intentionally separated. A historical aggregate is not added to the latest scoped matrix, because many cases overlap and use different harnesses.

30.1 Historical aggregate snapshots

- 302/302: the recurring full task-suite snapshot during the multi-domain and multilingual expansion.

- 305/305: the later local snapshot recorded after the empty-citation grounding repair and related missed-period correction.
- 250-query evaluation: 220 reference questions plus 30 Hindi transliteration questions, reporting 97.2% routing accuracy, 100% safety, and 100% fallback behavior in that evaluation snapshot.

30.2 Current integrated backend matrix

The 18 July integrated-source verification ran the maintained suites separately: latency/context 14/14; focused profile validation 6/6; task suites 3 through 8 at 32/32, 31/31, 79/79, 76/76, 44/44, and 43/43; the emergency, follow-up, health-router, controller-static, and semantic-intent suites; and WhatsApp/Interakt 34/34.

30.3 Golden retrieval

The deterministic golden retrieval gate passed 30/30 with mean recall 1.000 in the final integrated check. During latency tuning, smaller candidate windows were rejected when they degraded that gate. A simulated rewrite-skip variant had also demonstrated that the gate detects evidence drift rather than merely checking process success.

30.4 WhatsApp count evolution

WhatsApp controller coverage grew from 33/33 during health-mode gating to 34/34 after the Health Query entry prompt and related orchestration behavior were added. The current number is 34/34; 33/33 is retained only as historical sequence evidence.

30.5 Live latency and routing samples

A seven-case controlled live suite passed all cases, followed by a five-case routing suite and a separate five-case real-history web-chat run. Together they covered fresh and repeated health questions, different questions in the same condition, emergency, trivia, ambiguous acknowledgement, unanchored acknowledgement, contextual follow-up, and false-positive health guards.

30.6 Frontend evidence

The current frontend passed a clean production build and static bundle checks. It uses /ask, excludes /ask/stream, and carries fresh coordinate fields. There are no automated frontend test files, so this evidence is build and static-path verification rather than a component or browser test suite.

31. Evaluation Methodology

31.1 Routing evaluation

Routing questions are labeled by intended condition, safety class, or fallback behavior. Evaluation measures whether the system reaches the correct branch before judging writing style. This catches an answer that sounds helpful but came from the wrong knowledge namespace.

31.2 Safety evaluation

Safety evaluation contains true positives and benign negatives. It checks not only that chest pain or sudden one-sided weakness escalates, but also that heart-health education, speech therapy, past facial weakness, ordinary headache, and health questions with trivia-like words do not trigger the wrong deterministic path.

31.3 Retrieval evaluation

The golden set stores validated rewritten queries and expected chunk identifiers. Using stored rewrites separates retriever quality from non-deterministic wording changes in a fresh rewrite call. Recall is computed per question, and the process exits unsuccessfully when the configured gate is not met.

31.4 Conversation evaluation

Conversation tests preserve turn order and question count. They verify that a one-word answer is interpreted from the previous assistant turn, that uncertainty changes the next action, and that different questions in the same condition do not receive a cached replay.

31.5 Grounding evaluation

Grounding tests exercise low confidence, malformed JSON, missing citations, high confidence with citations, low reply-context cosine, emergency backstop, and clarify-mode exception behavior. The missed-period regression is a specific guard against treating an empty citation array as automatic proof of an ungrounded answer.

31.6 Channel evaluation

Channel tests separate transport ownership from health behavior. WhatsApp cases cover signature decisions, fast acknowledgement, menu entry and exit, silence outside the health lane, one-time operational notices, human support, and the shared chatbot call. Web checks separately prove which endpoint the current frontend selects.

32. Before-and-After Evidence

- Answer grounding — before: free-form responses could reach users without a dependable structured or objective evidence gate.
- Answer grounding — after: strict schema parsing, confidence threshold, citation-aware high-confidence rule, and objective cosine verification.
- Clinical scope — before: one primary condition and English-centric retrieval.
- Clinical scope — after: six isolated domains, multi-condition routing, and multilingual embeddings.
- Emergency behavior — before: no dependable pre-model category response.
- Emergency behavior — after: category-specific medical responses, generalized neurological logic, mental-crisis separation, and false-positive guards.
- Reranking — before: a wrapper path could throw or return saturated scores without demonstrating separation.
- Reranking — after: direct cross-encoder inference, raw logits, known-pair separation proof, measured CPU strategy, and golden-gated window size.
- Caching — before: process-local maps with weak conversational scope.
- Caching — after: Redis-backed rewrite, embedding, and versioned response caches with user, namespace, age, page, and dialogue-state boundaries.
- Personal context — before: assistant text could enter personal retrieval and stale memories could be asserted as current facts.
- Personal context — after: user-message-only vectors, conservative legacy handling, 0.75 relevance filtering, topic-aware history, and anti-assumption prompts.
- Short replies — before: message length could decide follow-up status and a simple 'yes' could attach to the wrong implied cause.
- Short replies — after: structural 0/1/2+ question handling with deterministic clarification for ambiguous or unanchored acknowledgements.
- Normal uncached latency — before: reported responses could exceed 15 seconds and stage attribution was incomplete.
- Normal uncached latency — after: controlled samples completed in 5.666–8.716 seconds, with structured stage logs and safe exact-repeat responses near 0.161 seconds.
- WhatsApp ownership — before: the workflow and AI could both answer greetings, buttons, and free text.
- WhatsApp ownership — after: strict Health Query lane gating, silence in workflow-owned lanes, and one shared health-answer core.
- Web streaming claim — before documentation: backend availability and current frontend use were described as if they were the same state.
- Web streaming claim — corrected: the backend route remains available, while the verified current frontend uses /ask without SSE.

33. Engineering Evolution

The system evolved through a series of independently verifiable capability groups. This chronology focuses on technical state changes and excludes project administration.

- Grounded PCOS foundation: native vector retrieval, structured answer schema, confidence and cosine gates, emergency detection, query rewrite, hybrid retrieval, cache helpers, context tagging, and initial security review.
- Multi-domain and multilingual expansion: six namespaces, BGE-M3 embeddings, self-hosted reranking, condition routing, mental-wellness safety, personal memory, and multilingual evaluation.
- Conversation-quality repair: broader health intent, condition-aware emergency wording, uncertainty clarify mode, conversational voice, emotional support routing, personal-memory provenance, and answer-first prompting.
- Channel integration: backend SSE capability, Redis-backed repeated-response acceleration, WhatsApp transport, shared pipeline adapter, provider workflow ownership, health-mode gating, and one-time channel notices.
- July latency and context repair: structural acknowledgement state, versioned cache scope, assistant-text removal from personal evidence, stage timing, parallel independent work, measured reranker tuning, final-answer model selection, and sub-10-second controlled normal turns.
- July routing hardening: generalized stroke-like phrasing, broad attribution-trivia fast paths, and zero-question acknowledgement clarification.
- July source reconciliation: the latest editable frontend and backend histories were compared, combined in clean integration workspaces, built, and run through the maintained regression matrix. Current frontend endpoint behavior and backend capabilities were then documented separately.

34. Incident Chronology and Engineering Lessons

The following incidents are included because they explain why the current architecture has certain gates. Details are limited to reusable engineering lessons and verified behavior; private operational information is excluded.

34.1 Reranker appeared healthy but did not rank

The rerank wrapper threw on paired input in the deployed transformer runtime and silently fell back. When it did return scores, a classification transform compressed them near the same value. The repair used the tokenizer and sequence-classification model directly, read raw logits, and added a known relevant-versus-irrelevant separation probe. Lesson: component availability is not evidence of component usefulness.

34.2 Embedder fallback was invisible until traffic

The multilingual embedder loaded lazily, so an incompatible fallback dimension could remain hidden until a user asked the first question. Boot warmup made the active embedding mode and vector length visible before traffic. Lesson: warm every model whose fallback can violate a downstream data contract.

34.3 Crisis language reached the wrong emergency tone

Suicidal-ideation phrases were originally captured by the general medical emergency list before the compassionate crisis layer. Pure ideation phrases were moved to the crisis detector, while acute medical consequences remained medical emergencies. Lesson: detector ordering is a user-safety decision.

34.4 Empty citations rejected grounded missed-period answers

The model sometimes returned a high-confidence, corpus-supported answer with an empty citation array. The first gate rejected it automatically, producing repeated fallbacks for a core health question. The repair made low confidence the first rejection rule and forced objective cosine grounding whenever citations are absent. Lesson: metadata-format compliance and evidence quality are related but not identical signals.

34.5 Uncertain follow-ups repeated remedies

An uncertain reply recovered the same condition, retrieved the same chunks, and encouraged the same remedy list. A response cache could then replay nearly identical text. The repair added a clarify turn, excluded contextual follow-ups from response caching, and allowed question-only clarify output to bypass ordinary RAG similarity. Lesson: conversation progress needs an explicit state transition.

34.6 Test cleanup removed an unrelated record

A broad numeric-range deletion intended for temporary identities also matched one pre-existing record. Future live harnesses switched to exact created-identifier deletion and verified the exact count afterward. Lesson: test-data cleanup in a shared data store must be allowlisted, never range-based.

34.7 Personal memory invented user history

Old assistant prose and loosely related memories re-entered retrieval, leading the response to assert unstated breastfeeding, dizziness, or other background. The repair embedded only user-authored messages, reconstructed safe legacy records, omitted ambiguous legacy text, applied a 0.75 relevance gate, and limited prompt use to one directly relevant memory. Lesson: provenance must be enforced before semantic relevance.

34.8 Query-rewrite skipping changed evidence

A regex tried to avoid rewrite calls for self-contained questions. Chunk comparison showed that terminology normalization changed top results for several questions. Always-on rewrite was restored for the website, and the golden retrieval gate was added to prevent a resource optimization from silently changing evidence. Lesson: measure retrieved identifiers, not only answer fluency.

34.9 Two systems answered the same WhatsApp message

The provider workflow and the AI both answered greetings, menu steps, and free text. Health Query on/off markers and strict default silence established one owner per lane. Lesson: multi-channel orchestration needs explicit ownership, not merely webhook connectivity.

34.10 A one-time channel notice repeated forever

The notice path returned without recording that it had already fired, so every later inbound message satisfied the same condition. An append-only operational marker made the transition durable and excluded the marker from answer counts and model history. Lesson: notification delivery is state, not just output.

34.11 Short-message logic disabled cache and corrupted context

A message-length heuristic treated many ordinary questions as contextual follow-ups and disabled cache reuse. A multi-question assistant turn followed by 'yes' could also attach the answer to an invented cause. Structural acknowledgement rules, latest-turn stage detection, and versioned response scope repaired both integrity and latency. Lesson: dialogue structure is more reliable than character count.

34.12 Editable source histories diverged

The compiled live frontend, editable live-source line, and newer frontend history were not the same artifact. The backend also had newer live behavior and separate application changes. The resolution compared ancestry and file content, combined both valid sides in clean workspaces, preserved the original folders, built the frontend, and ran the full backend matrix. Lesson: a running bundle is deployment evidence, not automatically the authoritative editable source.

35. Deployment Topology and Operational Verification

The production topology uses a reverse proxy in front of a Node.js application managed by a process supervisor. The same backend serves broader application routes and the health assistant. Redis, MongoDB, the vector store, the local transformer runtime, the language-model

service, and the messaging provider are external or auxiliary dependencies around that process.

Logical production flow:

```
Browser / messaging provider
→ reverse proxy
→ Node.js application process
→ chatbot controller and services
→ data/model dependencies
→ controlled response through originating channel
```

35.1 Local-first change discipline

Runtime changes are prepared and tested locally before deployment. Relevant live files are compared with the known source state to detect drift. Files with unrelated live changes are patched narrowly rather than overwritten.

35.2 Backup and parity

A rollback copy is created before material runtime changes. After transfer, syntax checks run in the target environment and cryptographic hashes confirm that the intended local artifact and deployed file are identical.

35.3 Restart and health checks

The process is restarted once with current configuration, then checked for online state, stable restart count, and absence of a crash loop. Health checks cover the changed assistant path and representative unchanged application routes.

35.4 Controlled behavior verification

A deployment is not accepted on process health alone. The changed behavior is exercised through the real route, with deterministic controls for emergency and off-topic behavior, latency-stage inspection, and exact cleanup of temporary test records.

36. Verified Limitations and Risk Register

- No frontend automated suite: the web source builds successfully and static endpoint checks pass, but component, browser, accessibility, and interaction regressions are not protected by repository tests.
- SSE not selected by current web client: the backend path exists and has shared finalization, but current user-facing verification is for /ask. A future switch requires renewed frontend and end-to-end coverage.

- Safety patterns are not exhaustive: novel wording, typos, unsupported languages, indirect statements, and cultural phrasing can evade deterministic patterns. Model backstop behavior is not complete recall.
- Mental-health support is not clinical care: the crisis path provides support and escalation information but cannot assess immediate real-world risk like a trained responder.
- Model confidence is self-reported: the 0.60 and 0.90 thresholds are useful controls but not calibrated medical probabilities. Objective grounding and deterministic safety remain necessary.
- Grounding check has an availability tradeoff: if the extra embedding operation itself fails, the current finalizer logs the issue and can continue under the remaining signals rather than failing every answer.
- Corpus size is a documented snapshot: 346 chunks describes the verified ingestion state, not a promise that content never changes. Corpus edits need fresh counts and evaluation.
- Personal vector retention needs policy: per-user namespaces accumulate relevant turns and do not currently expose a documented automatic pruning or retention lifecycle at the vector layer.
- Fallback embeddings can violate the index contract: the 1024-dimensional corpus requires the multilingual embedder. Warmup exposes a fallback, but the operational response should be treated as degraded until dimensional compatibility is confirmed.
- External latency varies: normal requests still depend on model and vector services, network conditions, and local inference load. Observed sub-10-second samples are not a universal guarantee.
- BM25 warmup is eventually ready: dense retrieval remains active during keyword-index startup, so early requests may not use the full hybrid path.
- Product matching is intentionally simple: explicit product queries use keyword matching and a small result list. Relevance is not equivalent to clinical recommendation and should remain isolated from medical grounding.
- WhatsApp state reuses history markers: this avoids a schema change but couples channel state to bounded history and timeout logic. Dedicated state storage would be clearer at larger scale.
- Language coverage is uneven: multilingual embeddings provide broad semantic support, but the dated evaluation emphasizes English and Hindi transliteration rather than equally deep tests for every supported language.
- Security review is point-in-time: dependency changes, new routes, and new integrations require fresh review; an earlier clean review is not a permanent guarantee.

37. Recommended Next Engineering Gates

- Add frontend tests: cover composer request locking, coordinate refresh, image upload, authentication storage, error recovery, JSON responses, and any future SSE replacement event.

- Build a language-balanced evaluation: add native-script and transliterated cases for each priority language, with routing, retrieval, safety, and response quality labels.
- Add calibrated grounding evaluation: measure unsupported-claim rate and false fallback rate across citation-present and citation-absent cases instead of relying only on fixed thresholds.
- Make grounding failure policy explicit: decide whether a failed grounding-embedding call should fail closed, use a secondary verifier, or continue only for specific low-risk answer classes.
- Introduce corpus release metadata: record content version, chunk count, ingestion date, clinical reviewer, embedding model, and golden-evaluation result for every knowledge release.
- Create personal-memory retention controls: define maximum age, maximum records, deletion workflow, and user-visible data controls for personal vectors.
- Move channel state into a dedicated store: separate WhatsApp health-lane and one-time-notice state from conversational content once traffic or workflow complexity justifies it.
- Add provider-independent channel contracts: test inbound normalization and outbound delivery through fixtures so a provider payload change cannot silently alter health-lane ownership.
- Establish latency percentiles: aggregate stage logs into p50, p95, and p99 by cache status, route, channel, namespace, and provider outcome.
- Load-test local models: measure memory, CPU contention, warmup time, and concurrent rerank behavior before increasing traffic.
- Keep optimization behind retrieval gates: any rewrite skipping, smaller candidate window, alternative embedder, or reranker change must pass the golden set and the answer-level regression suite.
- Expand safety governance: review red-flag patterns and crisis wording with qualified clinical stakeholders, include novel paraphrases, and track false positives and false negatives over time.

38. Technology Stack and Responsibilities

38.1 Application

- Node.js and Express: HTTP routes, middleware, channel adapters, and orchestration.
- MongoDB with Mongoose: users, profile context, bounded history, flags, and surrounding application data.
- React frontend: web composer, location acquisition, authentication flow, chat display, and image input.

38.2 Retrieval and language intelligence

- Pinecone: condition namespaces and per-user vector memory.
- BGE-M3: 1024-dimensional multilingual embeddings for queries, history, memory, and grounding checks.
- BM25: per-namespace keyword retrieval over the vector corpus.
- MiniLM cross-encoder: local query-document reranking with raw-logit relevance scores.
- GPT-4o-mini: short self-contained retrieval-query rewriting.
- GPT-4.1-mini: strict structured final RAG answer generation.
- GPT-4o vision: supported image description for the optional image path.

38.3 Performance and operations

- Redis: rewrite, embedding, and response reuse with safe no-op degradation.
- Process supervisor: application lifecycle, online-state monitoring, and restart-count checks.
- Reverse proxy: public request routing to the application process.
- Structured logs: request and retrieval stage latency, outcomes, counts, and non-fatal degradation signals.

38.4 Channels

- JSON web endpoint: current verified frontend request path.
- SSE backend endpoint: available server capability using the shared finalizer, not selected by current frontend.
- WhatsApp provider integration: verified webhooks, quick acknowledgement, lane ownership, and outbound text delivery.

39. Current Status Summary

As of the latest evidence reconciliation, the core assistant is implemented across six clinical domains with multilingual retrieval, structured final answers, deterministic medical and mental-crisis paths, context-aware dialogue, relevance-gated personalization, Redis acceleration, JSON and backend SSE response modes, and a controlled WhatsApp health lane.

The current web frontend uses /ask and sends fresh coordinates when available. The backend /ask/stream capability remains present but is not selected by that frontend. The current final-answer model is GPT-4.1-mini; GPT-4o-mini is the query-rewrite model.

The current empty-citation behavior is objective verification, not automatic rejection: low confidence falls back; a citation-free answer must pass cosine grounding even at high self-rated confidence. This corrects the earlier description that every empty citation array immediately becomes a fallback.

The latest integrated backend verification passed the maintained routing, safety, context, personalization, task, WhatsApp, profile, and golden retrieval suites. The current frontend build passes, with existing warnings and no automated frontend tests.

Measured July samples show fast deterministic paths below one second, exact repeated answers near 0.161 seconds, and controlled normal health turns within 5.666–9.442 seconds. These results demonstrate the latency repair but remain samples under variable external conditions.

The strongest remaining engineering needs are frontend automation, broader language-balanced evaluation, explicit personal-vector retention, calibrated grounding assessment, aggregated latency percentiles, and recurring clinical and security review.

Appendix A — Representative Request Traces

A.1 Fresh grounded health question

input → safety clear → health route → raw cache miss
→ GPT-4o-mini rewrite → BGE-M3 embedding
→ dense + BM25 + personal query → cross-encoder top five
→ prompt + tagged context → GPT-4.1-mini structured answer
→ confidence/citation/cosine finalization → persist → cache → reply

A.2 Exact self-contained repeat

input → safety clear → acknowledgement clear → raw cache eligible
→ versioned user/context key hit → formatted cached reply
→ no rewrite, retrieval, rerank, or final generation

A.3 Ambiguous acknowledgement

'Yes' → inspect immediately preceding assistant turn
→ more than one question detected → deterministic clarify response
→ no cache, retrieval, or generation

A.4 Citation-free structured answer

structured JSON parsed → confidence ≥ 0.60 → citations empty
→ high-confidence shortcut disabled → embed reply and RAG context
→ cosine ≥ 0.55 accept; otherwise controlled fallback

A.5 Emergency

input → deterministic category match → fixed urgent response
→ optional safeguarding persistence → return
→ no normal query rewrite, retrieval, rerank, or generation

A.6 WhatsApp health question

verified webhook → immediate acknowledgement → parse event
→ active Health Query lane → shared chatbot in-memory adapter
→ same safety/RAG/grounding path → normalized outbound text

Appendix B — Glossary

- RAG — retrieval-augmented generation: retrieve evidence first, then generate an answer using that evidence.
- Dense retrieval — semantic vector similarity between query and document embeddings.
- BM25 — keyword-ranking algorithm based on term frequency, rarity, and document length.
- Cross-encoder — a model that scores a query and candidate passage together for direct relevance.
- Namespace — an isolated vector partition used here for a clinical domain or one user's memory.
- Structured output — model output constrained to a strict machine-validated JSON schema.
- Grounding — evidence that the generated answer is supported by retrieved context.
- Cosine similarity — a vector comparison used here for retrieval relevance, history filtering, memory filtering, and answer grounding.
- Fast path — a deterministic branch that returns before the full retrieval and generation pipeline.
- Graceful degradation — continuing with a defined reduced capability when an optional dependency fails.
- SSE — server-sent events, an HTTP mechanism for streaming incremental response events from server to browser.
- Golden retrieval set — a fixed set of questions and validated chunk identifiers used to detect evidence drift.
- Provenance — the origin of a fact, such as the current user message, stored user profile, retrieved corpus, or prior assistant text.

Appendix C — Review Checklist

For product review:

- Confirm the assistant's educational boundaries and escalation tone match the intended user experience.
- Confirm each clinical domain and language target has an accountable content-review process.

- Confirm backend capability and active frontend behavior are not conflated in release communication.
- Confirm limitations remain visible when presenting the system externally.

For engineering review:

- Trace a request through early safety, routing, cache eligibility, retrieval, finalization, and persistence.
- Verify any optimization against golden retrieval and conversation-state tests.
- Verify channel adapters call the shared health core and maintain one owner per interaction lane.
- Verify test cleanup uses exact created identifiers.
- Verify current frontend endpoint selection from source and built output.
- Verify model-role documentation matches runtime constants and service code.
- Verify citation-free answers are objectively grounded rather than automatically accepted or rejected.

For operational review:

- Warm local models before accepting traffic and inspect active dimensions.
- Confirm optional dependency failures degrade according to the documented contract.
- Inspect stage latency by cache state and route instead of relying on one average.
- Maintain rollback artifacts and deployed-file parity for runtime changes.
- Run unchanged-application smoke checks after assistant changes.

Developed by Zahidul Islam

Backend and AI engineering — multilingual retrieval, code-enforced safety, conversation integrity, operational efficiency, and multi-channel delivery.